

Designing for 99.99%

Availability math, SLA / SLO / SLI, RPO / RTO, MTTR, disaster recovery and cost, worked end to end on a financial app hosted on AWS

HOW TO READ THIS GUIDE

Every idea comes in small pieces: **plain meaning** → **analogy** → **the math, step by step** → **what it means for the design** → **the one-line interview answer**. Each part ends with a short self-check. Read one part per sitting; don't rush it.

Contents

1. The example: **PayKaro**, a payments app on AWS
2. The vocabulary: availability, the "nines", SLI / SLO / SLA, error budget
3. The failure clock: MTBF, MTTD, MTTA, MTTR
4. Losing data vs. losing time: RPO and RTO
5. Availability math: chains (serial) and backups (parallel)
6. The questions an architect asks before saying "yes, 99.99%"
7. The design: PayKaro on AWS, layer by layer
8. Disaster-recovery strategies and which one fits
9. The cost: infrastructure, people, and the business case
10. Interview questions with model answers
11. Back to GaramChai: the same ideas in our Kubernetes lab
12. Glossary

GaramChai K8s project · study guide · 2026-09-25

AWS SLA figures checked against [aws.amazon.com](https://aws.amazon.com/sla) SLA pages in September 2026. Prices are rough list prices for illustration only; always confirm with the AWS Pricing Calculator.

1. The example: PayKaro

We need one concrete app to hang every idea on. **PayKaro** (fictional) is an Indian payments app: customers pay shops by scanning a QR code, and the money moves from the customer's wallet to the merchant's.

Fact about PayKaro	Value (our assumption)	Why it matters
Transactions per day	5 million	Sizing (Part 5.5)
Busiest hour	7–9 pm (shopping, dinner)	Downtime at this hour hurts most
PayKaro earns per transaction	₹2 (fee)	Cost of downtime (Part 9)
Client's demand	99.99% availability	The whole guide
Where it runs	AWS, India	Indian rules on where payment data may be stored

WHY A PAYMENTS APP?

Because it makes every trade-off sharp. For a chai shop (GaramChai) an outage is annoying. For a payments app an outage means shoppers standing at a counter unable to pay, merchants losing sales, a regulator asking questions and a brand losing trust. And a lost transaction isn't lost chai, it's lost money.

2. The vocabulary

2.1 Availability

Plain meaning: the share of time the service works for its users.

Availability = time it worked ÷ total time

Example: a 30-day month has $30 \times 24 \times 60 = 43,200$ minutes.

The app was down for 43 minutes.

Availability = $(43,200 - 43) \div 43,200 = 0.99900 = 99.90\%$

Analogy: a shop's opening hours. If the shop promises to be open all day and is shut for 43 minutes in a month, it was "open 99.9% of the time".

2.2 The "nines": turning a percentage into minutes

Percentages hide how small the numbers get. Always convert them into **minutes**, which is what the business actually feels.

Allowed downtime = total time × (100 % - availability)

99.99 % over a 30-day month:

step 1: $100\% - 99.99\% = 0.01\% = 0.0001$

step 2: $43,200 \text{ minutes} \times 0.0001 = 4.32 \text{ minutes}$

Availability	"Nines"	per year	per 30-day month	per week	per day
99 %	two	3.65 days	7.2 h	1.7 h	14.4 min
99.5 %	two and a half	1.83 days	3.6 h	50 min	7.2 min
99.9 %	three	8.76 h	43.2 min	10.1 min	1.4 min
99.95 %	three and a half	4.38 h	21.6 min	5.0 min	43 s
99.99 %	four	52.6 min	4.32 min	1.0 min	8.6 s
99.999 %	five	5.3 min	26 s	6 s	0.9 s

Some sources use an average month of 30.44 days, so you'll also see "99% = 7.3 h per month". Both are fine; say which one you used.

THE ONE FACT THAT DRIVES THE WHOLE DESIGN

At 99.99% you may be down for **4 minutes a month**. A human paged at 3 am needs longer than that just to wake up, read the alert and log in. So at four nines, **recovery must be automatic**. Humans find the root cause afterwards; machines restore service.

2.3 SLI, SLO, SLA: three words people mix up

Think of a pizza shop that promises "delivered in 30 minutes or it's free".

Term	Plain meaning	Pizza analogy	PayKaro example
SLI Service Level Indicator	A measurement : a number you compute from real data	"How many minutes did each delivery actually take?"	% of payment requests that succeeded within 1 second
SLO Service Level Objective	An internal target for that measurement	"We aim for 99% of deliveries under 25 min"	99.99% of payments succeed within 1 s, over 30 days
SLA Service Level Agreement	A contract with a customer, with a penalty if missed	"Over 30 min: the pizza is free"	To merchants: 99.95% per month, or a fee refund

How they fit together

SLI = what you MEASURE (the speedometer)
 SLO = what you AIM for (your own speed rule: "I drive at 90")
 SLA = what you PROMISE (the legal limit: 100, with a fine)

Rule: SLA is looser than SLO. SLO 99.99 % → SLA 99.95 %
 The gap is your safety margin: you notice and fix things
 BEFORE you owe anybody money.

A good SLI is measured the way the user feels it

SLI (payments) = good payment requests ÷ all valid payment requests

"good" = answered without a server error (5xx) AND in under 1 second

"valid" = excludes requests the user got wrong (bad PIN, bad QR) → those are 4xx

A payment that "succeeds" after 40 seconds has failed for the shopper at the counter, which is why slow counts as bad.

TRAP: WHAT CLOUD SLAS REALLY PROMISE

AWS SLAs don't guarantee uptime. They define **service credits**: if a service falls below a threshold, you get a partial refund of *that service's* bill. Example: Aurora Multi-AZ gives a 10% credit below 99.99%. If PayKaro loses ₹50 lakh in an outage, AWS refunds a slice of the database bill. **Your SLA to your customers is your problem, not AWS's.** An SLA is a financial instrument, not an engineering guarantee.

2.4 Error budget: turning the SLO into a spending allowance

Plain meaning: if the target is 99.99%, then 0.01% of failure is **allowed**. That allowance is a budget you can "spend".

Time-based: $30 \text{ days} \times 0.01 \% = 4.32 \text{ minutes of downtime per month}$

Request-based: $200 \text{ M payments} \times 0.01 \% = 20,000 \text{ failed payments per month}$

PayKaro: $5 \text{ M/day} \times 30 = 150 \text{ M/month} \rightarrow 150 \text{ M} \times 0.0001 = 15,000 \text{ failures allowed}$

What it's for: it ends the endless fight between "ship features fast" and "don't break anything".

- Budget left → ship features, run experiments, take risks.
- Budget spent → **freeze risky releases**; the team works on reliability until the budget recovers.

Burn rate: how fast you're spending it

Burn rate 1 = spending exactly on plan (budget lasts the full 30 days)

Burn rate 10 = spending 10× too fast (whole budget gone in $30 \div 10 = 3$ days)

Alert rule used by many SRE teams (multi-window burn rate):

page a human if burn rate > 14.4 over the last 1 hour

($14.4 \times 1 \text{ h} = 2 \%$ of a 30-day budget gone in one hour)

INTERVIEW LINE

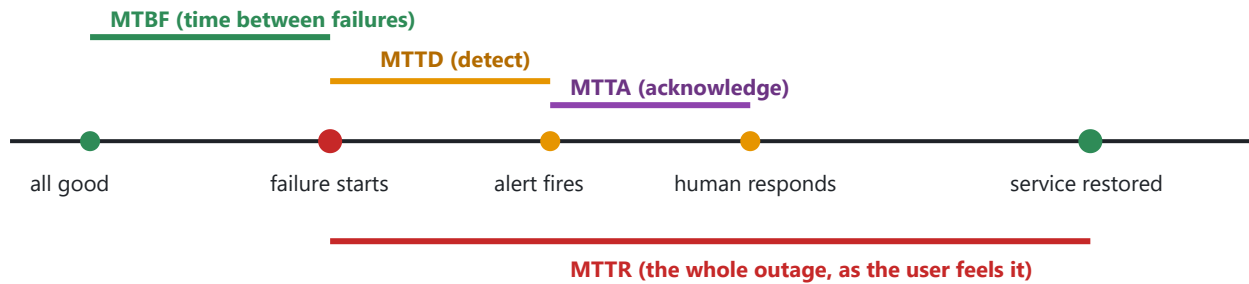
"SLIs are measurements from the user's point of view, SLOs are our internal targets, SLAs are contracts with penalties and are always looser than the SLO. The gap between 100% and the SLO is the error budget; when it's spent we slow releases and fix reliability."

Self-check

1. How many minutes per 30-day month does 99.95% allow? (Answer: $43,200 \times 0.0005 = 21.6$.)
2. Why should an SLA be looser than the SLO?
3. A payment that succeeds after 30 seconds: good or bad for the SLI? Why?

3. The failure clock: MTBF, MTTD, MTTA, MTTR

Every outage follows the same timeline. Each letter-soup term is just **one stretch of that timeline**.



Term	Stands for	Plain meaning	How to make it better
MTBF	Mean Time Between Failures	How long things usually work before the next failure. Bigger is better.	Redundancy, testing, safer deploys, fewer risky changes
MTTD	Mean Time To Detect	How long until anyone (or anything) notices	Good SLIs, synthetic checks every minute, burn-rate alerts
MTTA	Mean Time To Acknowledge	How long until a human says "I'm on it"	On-call rotation, clear paging, no alert noise
MTTR	Mean Time To Recover (also: Repair / Resolve)	From failure start to service working again. Smaller is better.	Automatic failover, one-click rollback, runbooks

"MTTR" is ambiguous: Recover, Repair, Respond and Resolve are all used. In an interview, say which one you mean. For availability, the one that matters is time to **recover service**.

3.1 The formula that connects them

$$\text{Availability} = \text{MTBF} \div (\text{MTBF} + \text{MTTR})$$

Plain words: of every "cycle" (working time + broken time), which share was working?

3.2 Worked example: how fast must PayKaro recover?

Suppose PayKaro has 1 incident per month.

$$\text{MTBF} \approx 30 \text{ days} = 43,200 \text{ minutes}$$

$$\text{Target availability} = 99.99 \%$$

$$0.9999 = 43,200 \div (43,200 + \text{MTTR})$$

$$43,200 + \text{MTTR} = 43,200 \div 0.9999 = 43,204.32$$

$$\text{MTTR} = 4.32 \text{ minutes} \quad \leftarrow \text{the whole incident, detect-to-recovered}$$

Now the human reality of ONE incident:

detect (alert needs a few minutes of bad data) 2 min

page reaches a sleepy engineer, acknowledged 5 min

open laptop, VPN, dashboards, understand 10 min

decide and act (failover / rollback) 10 min

total 27 min $\approx 6\times$ over budget

CONCLUSION

With humans in the loop, one incident a month is enough to miss 99.99%. There are only two ways out, and a 99.99% design uses **both**:

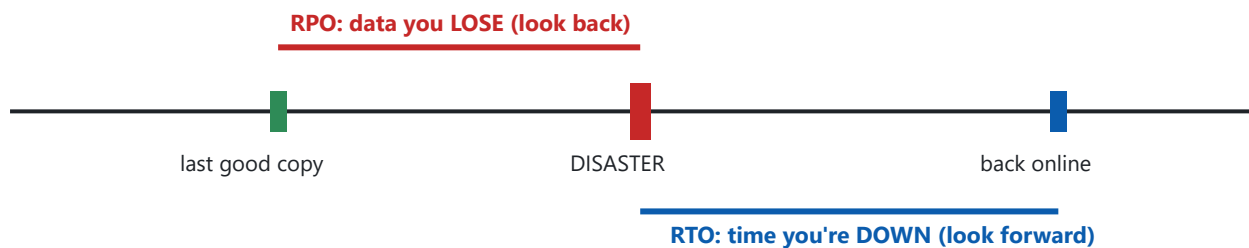
1. **Recover automatically in seconds** (health checks + automatic failover + automatic rollback), so MTTR is small.
2. **Fail less often** (redundancy, safer deploys), so MTBF is big.

Self-check

1. If PayKaro had 4 incidents a month, how long could each one last at 99.99%? (Answer: $4.32 \div 4 \approx 65$ seconds.)
2. Which of MTTD / MTTA / MTTR does automatic failover shrink?

4. Losing data vs. losing time: RPO and RTO

Availability asks "is it up?". Two more questions matter in a disaster: "**how much data did we lose?**" and "**how long until we're back?**"



Term	Question it answers	Analogy: writing a document	PayKaro target
RPO Recovery Point Objective	How much recent data may we lose? (measured as time)	You last pressed "Save" 10 minutes ago, then the laptop died: RPO = 10 minutes of typing lost	≈ 0 for the ledger (a lost payment is lost money); minutes for analytics
RTO Recovery Time Objective	How long may recovery take?	How long until you're typing again on another laptop	< 5 minutes for payments

4.1 What decides RPO: how the copy is made

Copy method	How it works	RPO	Price
Nightly backup	Copy once a day	up to 24 h	cheap
Point-in-time recovery	Backup + continuous transaction log	about 5 min	cheap
Asynchronous replication	Primary says "saved" first , sends the copy a moment later	seconds (whatever was in flight)	medium; no slowdown
Synchronous replication	Primary says "saved" only after the copy confirms	0	every write waits for the other side, so it's slower the farther away the copy is

THE PHYSICS YOU CAN'T NEGOTIATE WITH

Synchronous copies inside one AWS region (between Availability Zones, a few km apart) cost about a millisecond per write, which is fine. Synchronous copies between **regions** hundreds of km apart add tens of milliseconds to **every** write. That's why the common pattern is **synchronous inside a region, asynchronous between regions**, and you design around the few seconds of cross-region RPO (Part 7.4).

4.2 Related terms you'll hear

- **MTD / MTPD** (Maximum Tolerable Downtime / Period of Disruption): the point where the business suffers serious damage. RTO must be comfortably below it.
- **Durability vs. availability:** durability = "will my data still exist?"; availability = "can I reach it right now?". Amazon S3 Standard is designed for 99.999999999% (eleven nines) durability, but its availability is far lower (it's designed for 99.99%, and the SLA credit threshold is 99.9%). A service can be down while your data is perfectly safe.
- **BCP vs. DR:** the Business Continuity Plan covers the whole business (people, offices, suppliers). Disaster Recovery is the IT part of it.

INTERVIEW LINE

"RPO is how much data we can afford to lose, looking back from the disaster; RTO is how long we can afford to be down, looking forward. RPO is decided by how we copy data: sync gives zero, async gives seconds, backups give hours. RTO is decided by how automated the recovery is."

5. Availability math: chains and backups

5.1 Serial (a chain): every link must work

Analogy: Christmas lights wired in a chain. One bulb fails, the whole string goes dark. A request that passes through a load balancer, then the app, then the database needs **all three**.

Rule: multiply the availabilities.

```
Load balancer 99.99 % × App 99.99 % × Database 99.99 %  
= 0.9999 × 0.9999 × 0.9999  
= 0.9997  
= 99.97 %           ← LOWER than every single part
```

KEY INSIGHT

A chain is always **weaker than its weakest link**. Every extra hop (another microservice, another external API) pulls the total down. Ten services at 99.9% in a chain give $0.999^{10} \approx 99.0\%$.

5.2 Parallel (backups): only one needs to work

Analogy: two lamps in a room. The room is dark only if **both** fail at the same time.

Rule: $\text{availability} = 1 - (\text{chance that ALL copies are down})$

Two copies, each 99 % (each down 1 % of the time):

chance both down = $0.01 \times 0.01 = 0.0001$

availability = $1 - 0.0001 = 0.9999 = 99.99 \%$

Two copies, each 99.9 %:

both down = $0.001 \times 0.001 = 0.000001 \rightarrow 99.9999 \%$

This is the magic of redundancy: two "ordinary" things in parallel beat one excellent thing.

BUT: THE TWO HIDDEN ASSUMPTIONS

1. **Independence.** The math assumes the copies fail *independently*. Two servers in the same rack, on the same power feed, running the same buggy release, fail **together**. That's why copies go in different Availability Zones (separate power, cooling, network) and different regions, and why a bad deploy is rolled out to one copy at a time.
2. **Instant failover.** The math assumes switching to the backup takes zero time. Real failover takes seconds to minutes, and **that switching time counts as downtime**. In practice MTTR of the failover, not the parallel formula, sets your real availability.

5.3 Putting both together (the method)

1. Draw the request path from the user to the data.
2. Components one after another → **multiply**.
3. Components that back each other up → **1 - (product of their unavailabilities)**.
4. Subtract the realistic **failover time**.
5. Compare with the target.

5.4 Worked example: PayKaro in ONE AWS region

Using AWS's published SLA credit thresholds (checked September 2026) as conservative numbers:

Component (Multi-AZ)	SLA threshold
Application Load Balancer (Multi-AZ)	99.99 %
EC2 instances across ≥ 2 AZs (EKS worker nodes; Region-Level SLA)	99.99 %
Aurora PostgreSQL, Multi-AZ cluster	99.99 %

One region = $0.9999 \times 0.9999 \times 0.9999 = 0.9997 = 99.97 \%$
= $43,200 \times 0.0003 \approx 13$ minutes down per month (on paper)
Target 99.99 % = 4.3 minutes $\rightarrow$ ONE REGION DOESN'T MEET IT ON PAPER

TWO HONEST CAVEATS

- **SLA numbers are pessimistic.** They're credit thresholds, not measured availability; real services usually do better. Composing SLAs gives a **conservative floor**, which is exactly what you want when you're signing a contract.
- **The Kubernetes control plane isn't in the request path.** EKS's standard control plane SLA is 99.95%, but if the API server is down, the pods keep serving traffic; you just can't deploy or scale for a while. So it doesn't go into the chain for "can a customer pay?". It *does* matter for your MTTR, because you can't roll back without it. (EKS also offers a Provisioned Control Plane tier with a 99.99% SLA.)

5.5 Worked example: PayKaro in TWO regions

Each region: 99.97 % $\rightarrow$ unavailable 0.0003 of the time

Both regions down at once (if independent):

$0.0003 \times 0.0003 = 0.00000009 \rightarrow$ availability 99.99999 % (on paper)

Reality check with failover time:

Assume a serious regional problem once a year, lasting 2 hours.

Single region: 120 min down > 52.6 min yearly budget $\rightarrow$ SLO BROKEN for the whole year

Two regions, automatic failover in 5 min:

5 min down $\approx 10 \%$ of the 52.6 min budget $\rightarrow$ SLO survives

CONCLUSION FOR PAYKARO

99.99% needs **at least two regions** with **automatic, practised failover**. Within each region, everything runs across **three Availability Zones**. That's the design in Part 7.

5.6 Sizing: how many transactions per second?

$5,000,000$ transactions $\div$ $86,400$ seconds per day = 58 per second (average)

Evening peak: assume $4 \times$ average = ~ 230 per second

Design target with headroom = 300 per second

This is Little's-law style thinking again: average rate first, then a peak factor, then headroom.

Self-check

1. Four services in a chain, each 99.9%. Total? ($0.9994 \approx 99.6\%$)

2. Two independent copies, each 99.5%. Total? ($1 - 0.005 \times 0.005 = 99.9975\%$)
3. Why does "two copies in the same rack" break the parallel formula?

6. The questions an architect asks before saying "yes, 99.99%"

The best first answer to "we need 99.99%" isn't "yes". It's a set of questions, because each extra nine costs roughly 10× more.

6.1 Business questions (always first)

Question	Why it matters	PayKaro answer (example)
What does 1 hour of downtime cost ?	Sets how much reliability is worth paying for	≈ ₹16.6 lakh at peak, plus penalties and reputation (Part 9)
Which user journeys need 99.99%?	Not everything needs the same level	"Pay" and "check balance": 99.99%. Statements, offers, profile: 99.9%
When? 24×7 or business hours?	Changes the whole budget	24×7 (people pay at night too)
Is planned maintenance downtime allowed?	If not, all changes must be zero-downtime	No
What are the RPO / RTO per data type?	Drives replication choices	Ledger RPO ≈ 0, RTO < 5 min
Which regulations apply?	Can force where data lives and how DR is tested	Indian payment-data storage rules, PCI DSS if card data is handled (confirm with compliance)
How will availability be measured and reported ?	Avoids arguments later	Request-based SLI from the load balancer + synthetic checks

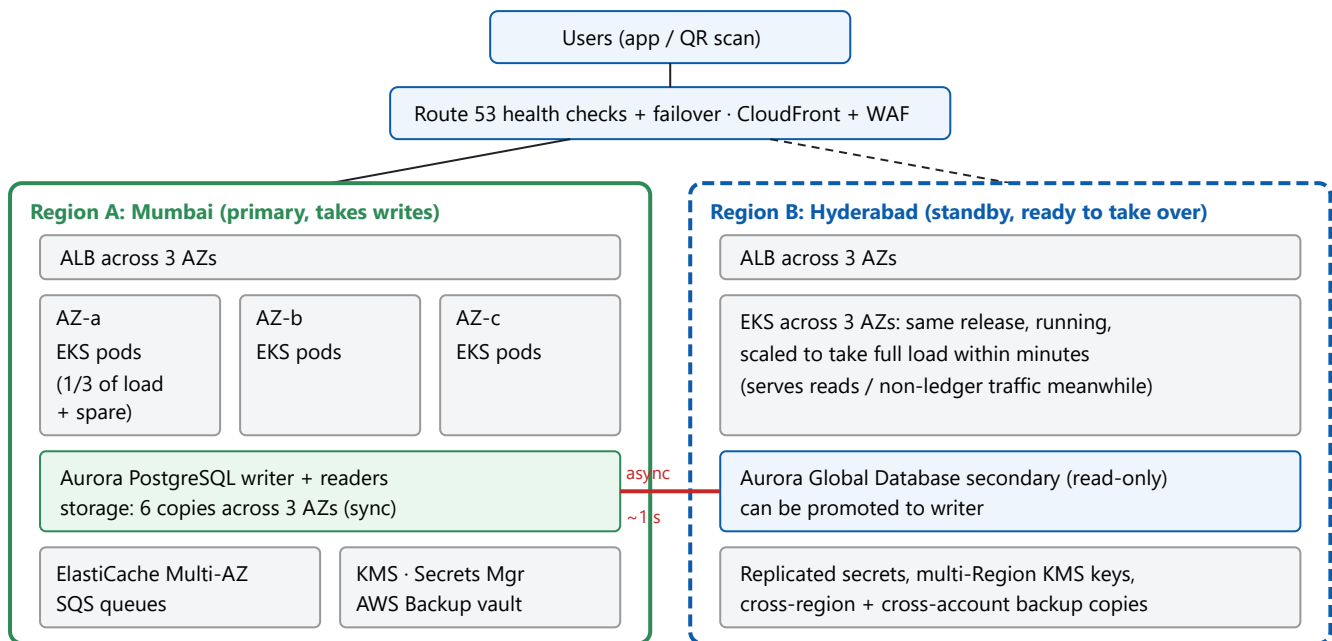
6.2 Technical questions

- **Single points of failure:** is there anything with only one copy? (one database writer, one NAT gateway, one DNS provider, one person who knows the system)
- **Dependencies:** list every external service in the payment path (bank/UPI switch, SMS OTP, fraud check). What's *their* SLA? What does PayKaro do when one is down: fail, queue, or degrade?
- **Data consistency:** can two regions accept writes for the same account? (For a ledger: usually **no**, because two regions both debiting the same wallet is how money gets double-spent.)
- **Failure detection:** how fast do we know, and who decides to fail over: a machine or a person?
- **Changes:** most outages are caused by our own deployments and configuration changes. How are they rolled out and rolled back?
- **Capacity:** if one AZ (or one region) disappears, can the rest carry 100% of the load *immediately*?

6.3 People and process questions

- Is there a **24×7 on-call rotation**? (Needs about 5–8 people so nobody is on call every week.)
- Are there **runbooks**, and have failovers been **practised** (game days)?
- Is there a **blameless post-incident review** after every outage?
- Who owns the **error budget** decision to freeze releases?

7. The design: PayKaro on AWS, layer by layer



7.1 Where the data lives (regions)

Two Indian regions: **Mumbai (ap-south-1)** as primary and **Hyderabad (ap-south-2)** as the second region. Keeping both in India matters because the Reserve Bank of India's 2018 directive requires payment-system data to be stored in India; always confirm the current rules with your compliance team.

7.2 Front door: DNS and edge

- **Route 53** with **health checks** and **failover routing**: if Mumbai's health check fails, DNS sends users to Hyderabad. Keep DNS TTLs short (for example 60 s) so clients follow quickly. (Route 53's SLA credit threshold is 100%: it's one of the most available services AWS has.)
- **Route 53 Application Recovery Controller (ARC)**: routing controls you flip deliberately in a regional event, so failover is one reliable action instead of a guess.
- **CloudFront + AWS WAF + Shield**: absorbs traffic spikes and attacks before they reach the app.

7.3 Compute: EKS across three Availability Zones

- Every service runs **several replicas spread across 3 AZs** (Kubernetes topologySpreadConstraints).
- **Static stability**: if one AZ disappears, the other two must carry 100% of the load **without waiting for autoscaling**. So each AZ runs at most about two-thirds full. That spare capacity is the price of not depending on "scale up during the emergency".
- **Readiness probes** take a sick pod out of the load balancer in seconds; **liveness probes** restart hung ones; **PodDisruptionBudgets** stop maintenance from taking too many replicas down at once.
- **HPA** (pods) and **Karpenter / Cluster Autoscaler** (nodes) for normal growth.

7.4 Data: the hardest part

- **Aurora PostgreSQL** in Mumbai: storage keeps **6 copies across 3 AZs**, and a write counts only when 4 of the 6 confirm. Losing a whole AZ loses **no committed data** (RPO 0 inside the region), and a reader is promoted to writer automatically.

- **Aurora Global Database** copies to Hyderabad **asynchronously**. AWS states typical cross-region lag under 1 second, an RPO of about 1 second, and promotion of the secondary region in under 1 minute.
- **One writer region at a time** for the ledger. Two regions writing the same wallet would risk double-spending, so the design is "active-active for reads, active-passive for writes".
- **The ~1 second gap on regional failover:** a few in-flight payments may not have reached Hyderabad. The mitigations:
 - **Idempotency keys:** every payment request carries a unique ID; retrying the same ID never charges twice.
 - **Reconciliation:** after failover, compare with the bank/UPI switch's records and repair the few affected transactions.
 - **Outbox pattern:** the ledger write and the "send notification" event are saved in one transaction, so events are never lost or invented.
- **Backups still matter:** replication faithfully copies mistakes too (a bad migration, a deleted table, ransomware). Use **point-in-time recovery** plus **AWS Backup** copies to another region **and another account**, with **Vault Lock** so nobody, including an attacker with admin rights, can delete them.

7.5 Resilience patterns in the code

Pattern	Plain meaning	Protects against
Timeouts	Never wait forever for another service	One slow dependency freezing everything
Retries with backoff + jitter	Try again, waiting a bit longer (and a bit random) each time	Short blips, without a retry storm
Circuit breaker	After repeated failures, stop calling for a while and fail fast	Hammering a service that's already down
Bulkheads	Separate pools per dependency, like watertight compartments in a ship	One bad dependency using up all threads/connections
Graceful degradation	Turn off non-essential features, keep "Pay" working	Partial outages becoming total ones
Load shedding	Politely reject excess requests instead of collapsing	Traffic spikes
Cell-based architecture	Split users into independent "cells" (for example by merchant ID)	One failure or bad deploy reaching everyone ("blast radius")

7.6 Changes: where most outages come from

- **Canary releases:** send 1% → 10% → 50% → 100% of traffic to the new version, watching the SLIs at each step; **roll back automatically** if error rate or latency rises.
- Deploy **one region at a time** (and one cell at a time), never everywhere at once.
- **Feature flags:** turn a feature off in seconds without a deploy.
- Database migrations in **backward-compatible steps** (expand → migrate → contract), so the old and new versions can run side by side.

7.7 Seeing problems: observability

- **The four golden signals** per service: latency, traffic, errors, saturation.
- **Synthetic checks** (for example CloudWatch Synthetics) make a test payment path every minute from outside, so you learn about problems before customers do.
- **SLO burn-rate alerts** (Part 2.4) instead of hundreds of noisy CPU alerts.

7.8 Practice: failures you rehearse don't surprise you

- **Game days:** planned regional failover drills (quarterly), with the real on-call team.
- **Chaos engineering** with **AWS Fault Injection Service**: kill pods, stop an AZ's instances, add latency, and check that the SLO holds.
- **Restore drills:** prove backups restore, just like our etcd drill in the GaramChai lab.

8. Disaster-recovery strategies and which one fits

AWS describes four classic strategies. Think of them as **how ready your second site is**.

Strategy	What the second region has	Analogy	RPO	RTO	Cost
Backup & restore	Only backups. Build everything after the disaster	Spare keys at a friend's house, but no spare car	hours	hours to a day	\$
Pilot light	Data replicated; servers defined (IaC) but switched off	Gas stove with only the small pilot flame burning	seconds–minutes	tens of minutes	\$\$
Warm standby	Data replicated; a smaller working copy running	A second shop open with one cook; call in more staff	seconds	minutes	\$\$\$
Multi-site active/active	Full copy running and serving traffic	Two full shops open, both serving customers	near 0	near 0	\$\$\$\$

WHICH FITS PAYKARO?

99.99% (4.3 min/month) rules out backup & restore and pilot light: their RTO is longer than the whole monthly budget. The realistic choice is a **hot/warm standby that's already running and already scaled** (the Part 7 design), or full active/active for stateless and read traffic with a single write region for the ledger. RTO target: a few minutes, automated, practised.

9. The cost

READ THIS FIRST

The numbers below are **rough, illustrative on-demand list prices** (approximately us-east-1, 2025–26; Mumbai differs a little), without discounts, data transfer or taxes. Their purpose is to show **proportions**, not to quote a price. For a real estimate use the AWS Pricing Calculator.

9.1 Infrastructure: three levels of ambition

Item (monthly, rough)	A: 99.9% 1 region, 3 AZs	B: 99.99% + warm standby region	C: 99.99%+ active/active, full size
EKS control plane(s) (~\$73 each)	\$73	\$146	\$146
Worker nodes (m6i.xlarge ≈ \$140 each)	6 → \$840	9 + 6 → \$2,100	9 + 9 → \$2,520
Aurora (r6g.large ≈ \$190 each) + storage/I/O	2 → \$480	2 + 1 → \$770	2 + 2 → \$960
Global Database replication + cross-region transfer	–	~\$200	~\$300
ElastiCache, ALB, NAT gateways (3 per region)	~\$450	~\$900	~\$900
Monitoring, logs, synthetics, backups (cross-account)	~\$300	~\$600	~\$700
Infrastructure total (rough)	≈ \$2,100	≈ \$4,700	≈ \$5,500

Why 9 nodes instead of 6 in the primary for B and C? Static stability: after losing one of three AZs, the remaining two must still carry 100% (6 nodes of work ÷ 2/3 = 9 nodes).

9.2 The part people forget: people, support and process

Item	Rough monthly cost	Needed for
24×7 on-call SRE team, ~6 engineers (assume ₹30 lakh/year each)	≈ ₹15 lakh (≈ \$18,000)	99.99% (at 99.9% a smaller business-hours team can work)
AWS Enterprise Support (published minimum; check current terms)	≈ \$15,000	Fast AWS escalation during incidents
Observability / incident tooling (SaaS)	\$1,000–5,000	Detecting and managing incidents
Game days, chaos testing, audits	engineer time	Proving it works

THE SURPRISE

Going from 99.9% to 99.99%, the **infrastructure** roughly doubles (+\$2,600/month), but the **people and support** add around \$33,000/month. At four nines, **most of the cost is people and process, not servers.**

9.3 The business case: is it worth it for PayKaro?

Cost of 1 hour down at the evening peak:

$230 \text{ payments/s} \times 3,600 \text{ s} = 828,000 \text{ payments}$

$828,000 \times ₹2 \text{ fee} = ₹16.6 \text{ lakh lost fees per hour } (\approx \$20,000)$

+ merchant SLA refunds, regulator scrutiny, customers switching apps (hard to price, often bigger)

Worst case (all downtime at peak):

99.9 % → 43 min/month × ₹16.6 lakh/60 ≈ ₹11.9 lakh/month

99.99 % → 4.3 min/month ≈ ₹1.2 lakh/month

Difference ≈ ₹10.7 lakh/month (≈ \$13,000) of direct fees

Extra cost of 99.99 %: ≈ \$2,600 infra + ~\$33,000 people/support ≈ \$35,000/month

READING THE RESULT LIKE AN ARCHITECT

On direct fees alone, 99.99% doesn't pay for itself. What tips the decision for a payments company is what the spreadsheet can't easily hold: **trust** (a customer who can't pay twice may switch apps for good), **merchant contracts**, and **regulatory expectations**. So for PayKaro the answer is "yes, for the Pay and Balance journeys", and deliberately **not** for statements, offers and analytics, which stay at 99.9%. For GaramChai the same math says 99.5% is plenty: an hour down costs ≈ ₹5,000.

10. Interview questions with model answers

Question	Model answer (short)
A client asks for 99.99%. What do you do?	Ask what an hour of downtime costs, which journeys need it, when, the RPO/RTO per data type and the regulations. Then show the cost of each nine and recommend a level per journey.
How much downtime is 99.99%?	52.6 min/year, about 4.3 min per 30-day month, about 8.6 s per day.
SLI vs SLO vs SLA?	SLI = a user-centric measurement; SLO = internal target; SLA = contract with penalties, looser than the SLO.
What's an error budget and how is it used?	100% minus the SLO. While there's budget, ship; when it's spent, freeze risky changes and invest in reliability. Alert on burn rate.
RPO vs RTO?	RPO = data you can lose (look back from the disaster); RTO = time to recover (look forward). RPO comes from the replication method; RTO from automation.
How do you calculate the availability of a system?	Serial parts multiply; redundant parts: $1 - \text{product of unavailabilities}$; then subtract failover time and check the independence assumption.
Can one AWS region give 99.99%?	On SLA math, three 99.99% services in a chain give about 99.97%. Plus a regional event can exceed the yearly budget alone. For a hard 99.99% commitment: two regions with automatic failover.
Sync vs async replication?	Sync: RPO 0 but every write waits, fine within a region. Async: no write penalty but seconds of RPO, used across regions. Design around the gap with idempotency and reconciliation.
Why not active/active writes for a ledger?	Two regions accepting writes for the same account risk conflicts and double-spend; one writer region with fast promotion is safer.
What is static stability?	Pre-provisioned capacity so that losing an AZ or region needs no scaling action during the emergency.
Replication exists, so why keep backups?	Replication copies mistakes and attacks instantly. Backups (PITR, cross-account, immutable) protect against logical corruption and ransomware.
How do AWS SLAs work?	They're service-credit thresholds, not guarantees; a credit is a small refund, and it doesn't cover your losses.
What causes most outages?	Changes: deployments and configuration. Hence canaries, automatic rollback, one region/cell at a time, feature flags.
How do you know DR works?	Practised game days and restore drills, measured against the RTO/RPO; chaos experiments with AWS FIS.
Where does the money go at 99.99%?	Infra roughly doubles, but people (24×7 on-call), support and process dominate.

11. Back to GaramChai: the same ideas in our lab

Idea from this guide	PayKaro (AWS)	GaramChai (our kubeadm lab)
Availability target	99.99% for Pay	99.5% (3.6 h/month); ₹5,000 per hour down doesn't justify more
RPO / RTO	≈ 0 / < 5 min	Orders: 4 h / 1 h (Postgres backup CronJob every 4 h); etcd: 6 h
Redundancy	3 AZs × 2 regions	2 worker nodes; ≥ 2 replicas + topologySpreadConstraints (Phase 9)
Fast recovery (MTTR)	Automatic failover	Readiness/liveness probes, Deployment self-healing (Phase 3)
Safe maintenance	PDBs, one AZ at a time	PodDisruptionBudget + kubectl drain (Phase 10)
Safe changes	Canary + auto rollback	Rolling update + kubectl rollout undo (Phase 3)
Capacity	Static stability, Karpenter	requests/limits, HPA, 50-user load test (Phase 10)
Backups that are proven	AWS Backup + drills	etcd timer + restore drill (done ) , Postgres drill (Phase 6)
Honest single points of failure	Removed	1 control plane, 1 Postgres, NFS on cka-m: documented, accepted for a lab

12. Glossary

Term	One-line meaning
Availability	Share of time (or of requests) the service works for users
Nines	Shorthand for availability: 99.99% = "four nines"
SLI / SLO / SLA	Measurement / internal target / contract with penalties
Error budget	Allowed unreliability: 100% – SLO
Burn rate	How fast the error budget is being used; 1 = exactly on plan
MTBF	Average time between failures (bigger is better)
MTTD / MTTA	Average time to detect / to acknowledge an incident
MTTR	Average time to recover service (smaller is better)
RPO	Maximum data loss, measured as time before the disaster
RTO	Maximum time to restore service after the disaster
MTD	Maximum tolerable downtime before serious business damage
Durability	Probability data isn't lost (different from availability)
Availability Zone (AZ)	One or more separate data centres in a region, with independent power and network
Region	A geographic area with several AZs (Mumbai, Hyderabad)
Serial / parallel availability	Chain: multiply. Redundant copies: 1 – product of unavailabilities
Sync / async replication	Copy confirmed before / after the write is acknowledged
Static stability	Enough capacity pre-provisioned to survive a failure without scaling
Blast radius	How many users one failure or bad change can affect
Idempotency key	Unique request ID so a retry never charges twice
Circuit breaker	Stops calling a failing dependency for a while; fail fast
Canary release	New version to a small share of traffic first, rollback on bad SLIs
Game day / chaos engineering	Deliberately causing failures to prove recovery works
Backup & restore / pilot light / warm standby / active-active	DR strategies from cheapest and slowest to most expensive and fastest

Sources for AWS figures (checked Sept 2026): AWS SLA pages for EC2, ELB, RDS, Aurora, EKS, DynamoDB, Route 53, CloudFront; Aurora Global Database product page. SLAs and prices change: always re-check before quoting them to a client.